Reservas de hotel

Álvaro Martorell, Éden Meshulam, Víctor Suesta

Enero 2025

1. Introducción

El aprendizaje automático (*machine learning*) es una disciplina de la inteligencia artificial que se basa en el desarrollo de algoritmos capaces de aprender patrones a partir de datos y generalizar esos conocimientos a nuevas situaciones. Estos métodos permiten mejorar la toma de decisiones y automatizar tareas complejas en sectores muy diversos, como la salud, las finanzas, la educación y el turismo. En particular, en un mundo cada vez más orientado por los datos, el aprendizaje automático se ha consolidado como una herramienta indispensable para extraer conocimiento y guiar estrategias basadas en evidencias.

El sector turístico es uno de los pilares económicos globales y representa un gran volumen de información operativa. En este contexto, las reservas de hotel son una fuente rica de datos que pueden proporcionar conocimientos sobre el comportamiento de los clientes, las tendencias del mercado y los factores que determinan cancelaciones o estancias efectivas. Por lo tanto, analizar estos datos a través de algoritmos de aprendizaje automático puede ayudar a las empresas hoteleras a optimizar sus operaciones, reducir las cancelaciones y mejorar la experiencia del cliente.

1.1. Motivación y justificación de la elección del conjunto de datos

El conjunto de datos seleccionado para este trabajo corresponde a un conjunto de reservas de hotel obtenido de la plataforma Kaggle. Este dataset ha sido creado por Ashan Raza y contiene 36.275 observaciones con 19 variables, de las cuales se han seleccionado 9 para el análisis. Estas variables incluyen características como el tiempo de antelación en la reserva ("lead time"), el número de adultos, el precio promedio de la habitación ("avg price per room"), el estado final de la reserva ("booking status") y otros factores relevantes.

La elección de este conjunto de datos responde a varias motivaciones. Primero, su tamaño y diversidad proporcionan una base sólida para la aplicación de métodos de aprendizaje automático, ya que permiten identificar patrones significativos y desarrollar modelos predictivos fiables. Segundo, el problema abordado—la cancelación de reservas—es de alta relevancia para el sector turístico. Las cancelaciones no solo afectan los ingresos de los hoteles, sino también su capacidad para planificar recursos y optimizar la ocupación. Por lo tanto, comprender qué factores contribuyen a las cancelaciones puede tener un impacto directo en la eficiencia y la rentabilidad de las operaciones hoteleras.

Otro aspecto clave es la variabilidad en los tipos de datos incluidos. Este dataset combina variables numéricas y categóricas, lo que permite un análisis multifacético y la aplicación de diferentes métodos de aprendizaje automático. Además, los datos incluyen información tanto sobre características específicas de las reservas como sobre el comportamiento de los clientes, ofreciendo una perspectiva integral del problema.

1.2. Objetivos del trabajo

El objetivo principal de este trabajo es desarrollar un modelo predictivo que permita determinar la probabilidad de que una reserva de hotel sea cancelada, basándose en las características de la reserva y del cliente. Este objetivo se desglosa en los siguientes subobjetivos específicos:

- Realizar una exploración descriptiva de los datos para comprender la distribución de las variables principales y detectar posibles patrones, sesgos o valores atípicos que puedan influir en el análisis posterior.

- Identificar las variables con mayor correlación con el estado final de la reserva ("Canceled" o "Not Canceled") y determinar cuáles de ellas pueden ser utilizadas como predictores en el modelo.
- Aplicar métodos de aprendizaje automático, como árboles de decisión, regresiones logísticas y otras técnicas adecuadas, para construir modelos que permitan predecir con precisión el estado de las reservas.
- Interpretar los resultados obtenidos y ofrecer recomendaciones para las empresas hoteleras, enfocadas en reducir las cancelaciones y optimizar las estrategias comerciales.
- Proponer estrategias basadas en datos para la mejora de la experiencia de los clientes y la maximización de la rentabilidad hotelera.

En definitiva, este trabajo busca no solo obtener un modelo predictivo robusto, sino también proporcionar una comprensión profunda de los factores que influyen en el comportamiento de los clientes, y así ayudar a establecer estrategias empresariales basadas en datos.

2. Descripción del conjunto de datos

El conjunto de datos utilizado contiene un total de 36.275 observaciones con 9 variables principales seleccionadas para este análisis. Estas variables abarcan aspectos clave de las reservas, como el tiempo de antelación, el número de adultos y el precio promedio de las habitaciones, además de información sobre el estado final de la reserva y otros atributos significativos.

2.1. Estadísticas descriptivas de los datos

En la tabla de variables, se incluyen las siguientes descripciones:

- **Lead Time:** Indica el tiempo de antelación en días para realizar la reserva. Es una variable numérica que tiene un promedio de 85.23 días, con una varianza de 85.93.
- **Número de adultos:** Representa el total de adultos por reserva, con un promedio de 1.84 y una varianza de 0.51.
- **Precio promedio por habitación:** Muestra el precio medio por noche en USD, con un valor promedio de 103.42 y una varianza de 35.08.
- **Estado de la reserva:** Variable categórica ("Canceled" o "Not Canceled") que indica si la reserva fue cancelada o completada.
- **Segmento de mercado:** Identifica el canal de la reserva (por ejemplo, "Online" o "Offline").
- **Tipo de habitación reservada:** Describe el tipo de habitación elegida por el cliente.
- **Cliente recurrente:** Variable binaria que indica si el cliente es repetitivo (0 o 1).
- **Número de cancelaciones previas:** Refleja cuántas reservas anteriores fueron canceladas por el cliente.
- **Número de reservas previas no canceladas:** Indica las reservas anteriores completadas sin cancelación.

2.2. Importancia del conjunto de datos

La selección de este conjunto de datos permite abordar preguntas relevantes relacionadas con los factores que influyen en las cancelaciones de reservas. Por ejemplo, se investigó la relación entre el tiempo de antelación y las cancelaciones, así como el impacto del precio promedio en la probabilidad de cancelación. Además, se analizaron patrones específicos entre clientes recurrentes y el número de cancelaciones previas para identificar posibles estrategias de fidelización.

El conjunto de datos combina tanto información operacional (como precios y tiempos) como datos del comportamiento del cliente, lo que facilita un enfoque integral en el análisis predictivo.

3. Exploración de los datos

3.1. Estadísticas descriptivas de las variables principales

1. **lead_time**:
    - Media: 85.232557
    - Varianza: 85.930817
    - Distribución:

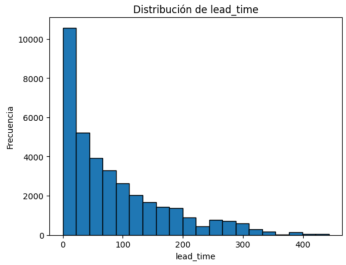


Figura 1:

2. **no_of_adults**:
    - Media: 1.844962
    - Varianza: 0.518715
    - Distribución:

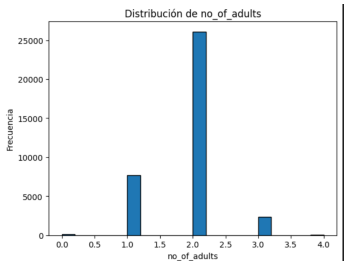


Figura 2:

3. **avg_price_per_room**:
    - Media: 103.423539
    - Varianza: 35.089424
    - Distribución:

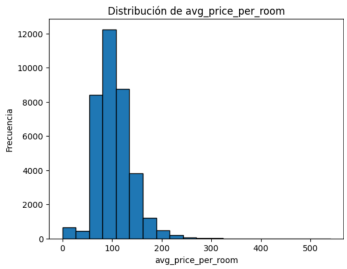


Figura 3:

4. **booking_status**:
    - Tipo: Categórica
    - Distribución:

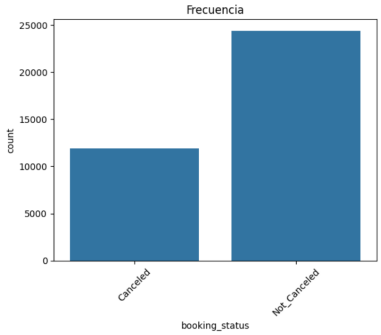


Figura 4:

5. **market_segment_type**:
    - Tipo: Categórica
    - Distribución:

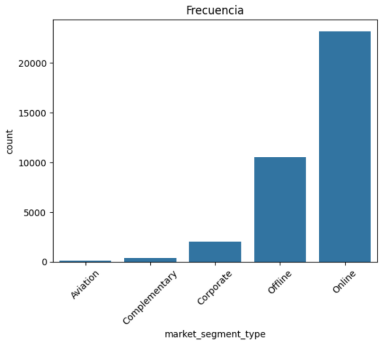


Figura 5:

6. **room_type_reserved**:
    - Tipo: Categórica
    - Distribución:

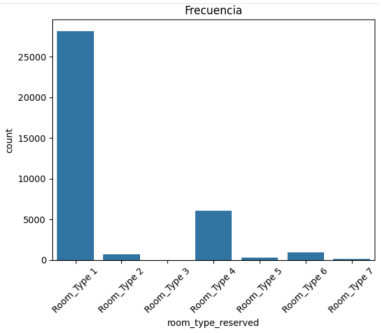


Figura 6:

7. **no_of_previous_cancellations**:

   - Media: 0.023349
   - Varianza: 0.368331
   - Distribución:

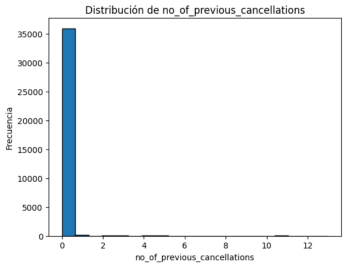


Figura 7:

8. **no_of_previous_bookings_not_canceled**:

   - Media: 0.153411
   - Varianza: 1.754171
   - Distribución:

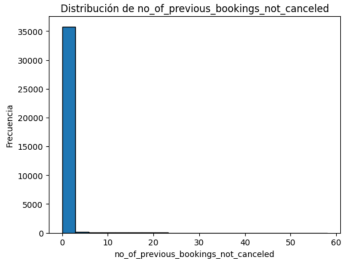


Figura 8:

3.2. Discusión de posibles sesgos o valores atípicos

La correlación positiva entre lead_time y booking_status muestra que cuanto mayor es el tiempo entre la reserva y la llegada, mayor es también la probabilidad de que la reserva sea cancelada
La Correlacion negativa entre no_of_special_requests y booking_status, en cambio, implica que los clientes con más solicitudes especiales tienden a cancelar menos sus reservas.
avg_price_per_room y booking_status tienen una correlación ligeramente negativa.Esto podría indicar que reservas más caras son menos propensas a ser canceladas, lo cual podria influir en estrategias de precios.
no_of_adults y avg_price_per_room también presentan una pequeña correlación positiva, pudiendo indicar que grupos más grandes tienden a gastar más. Esto puede ser útil para segmentar clientes.
Es óptimo seleccionar las variables con alta correlación con la variable objetivo (booking_status) para incluirlas como predictores en un modelo de aprendizaje automático. Eliminando de la misma manera las variables que no están correlacionadas del modelo para evitar ruido innecesario.

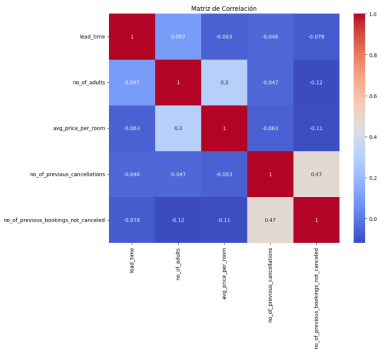


Figura 9:

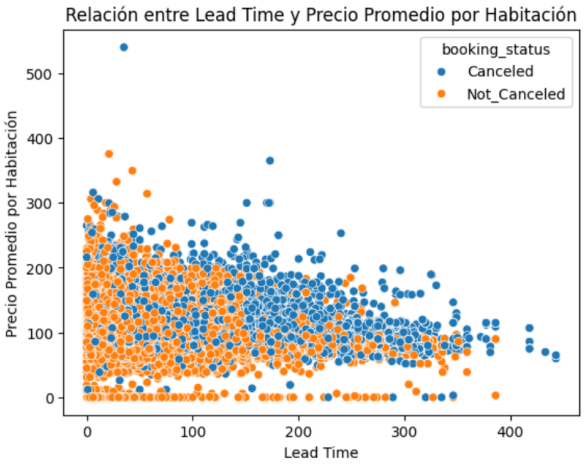


Figura 10:

Las reservas canceladas tienden a concentrarse en tiempos de reserva altos (lead_time > 100 días). Las cancelaciones (azules) parecen más frecuentes en reservas con precios promedio bajos (por debajo de 100).
Las reservas con alta anticipación (lead_time > 100 días) y precios bajos (avg_price_per_room < 100) son más propensas a ser canceladas.
Las reservas hechas con mucha anticipación son más propensas a ser canceladas. Esto podría deberse a cambios en los planes de los clientes
Los clientes que reservan habitaciones más baratas tienden a cancelar más, posiblemente porque tienen menos compromiso financiero o encuentran mejores opciones.
Utilizando el rango intercuartílico se han seleccionado las variables y casos con valores atípicos. Se han encontrado 1331 en lead_time y 812 en no_of_adults. Los outliers de la variable lead_time no se eliminaran (dado que son datos reales), podran ser analizados por separado o aplicar transformaciones (como logaritmo) para reducir su impacto en el modelo. En cambio, los valores en la variable no_of_adults como 10, 11 y 13 parecen no ser realistas, por lo que han sido eliminados (aquellos casos con mas de 6 adultos por reserva).

4. Algoritmos de aprendizaje automático

Hemos seleccionado diferentes algoritmos de regresión para estimar la variable objetivo, en este caso el precio medio por habitación, y comparar su rendimiento: Los algoritmos seleccionados abarcan desde modelos lineales hasta métodos que identifican patrones no lineales. Esto permite determinar el mejor enfoque para los datos, equilibrando simplicidad y precisión.

Estos son los algoritmos que hemos utilizado para modelar nuestra variable objetivo:

Regresión Lineal:

La regresión lineal es un modelo sencillo e interpretable que asume una relación lineal entre las variables predictoras y la variable objetivo. Se seleccionó como punto de partida por su simplicidad y facilidad de interpretación, permitiendo establecer una línea base para comparar el rendimiento de otros modelos más complejos.

Tras entrenar el modelo con los datos disponibles, el modelo de regresión lineal simple asume la siguiente forma:

$$\hat{y} = 10{,}398x_1 + 11{,}505x_2 - 0{,}328x_3 - 1{,}386x_4 - 2{,}509x_5 + 3{,}071x_6 - 0{,}493x_7 - 2{,}583x_8$$

Donde:

- $\hat{y}$: Precio estimado de una habitación de hotel.
- x_1: Número de adultos (`no_of_adults_scaled`).
- x_2: Número de niños (`no_of_children_scaled`).
- x_3: Número de noches entre semana (`no_of_week_nights_scaled`).
- x_4: Número de noches de fin de semana (`no_of_weekend_nights_scaled`).
- x_5: Tiempo de anticipación de la reserva (`lead_time_scaled`).
- x_6: Número de solicitudes especiales (`no_of_special_requests_scaled`).
- x_7: Número de cancelaciones previas (`no_of_previous_cancellations_scaled`).
- x_8: Número de reservas previas no canceladas (`no_of_previous_bookings_not_canceled_scaled`).

Elegimos este modelo como punto de partida por las siguientes razones:

- Proporciona una base sólida para evaluar la efectividad de modelos más complejos.
- Permite interpretar directamente la relación entre las variables predictoras y el precio medio, facilitando el análisis inicial de los datos.

Sin embargo, este modelo presenta limitaciones importantes, como la incapacidad de capturar relaciones no lineales y su sensibilidad a la multicolinealidad, lo que puede afectar su desempeño si las variables predictoras están correlacionadas.

Regresión Lasso y Ridge:

Estos algoritmos extienden la regresión lineal al incorporar regularización, lo que ayuda a prevenir el sobreajuste.

- **Lasso:** Penaliza coeficientes con L1 ($\lambda \sum |\beta_i|$), seleccionando las variables relevantes al reducir los coeficientes de las menos importantes a cero. La ecuación de la regresión Lasso ($\alpha = 1$) queda de la siguiente forma:

$$\hat{y} = 9{,}3761x_1 + 10{,}6053x_2 - 0{,}2909x_4 - 1{,}5496x_5 + 2{,}3738x_6 - 1{,}8142x_8$$

Esto aclara que las variables x_3 y x_7 han sido eliminadas debido a la regularización.

- **Ridge:** Penaliza con L2 ($\lambda \sum \beta_i^2$), que reduce la magnitud de los coeficientes pero no los elimina. Es útil para manejar problemas de multicolinealidad, ya que distribuye el peso entre variables correlacionadas. La ecuación de la regresión Ridge ($\forall \alpha$) queda de la siguiente forma:

$$\hat{y} = 10{,}3978x_1 + 11{,}5041x_2 - 0{,}3276x_3 - 1{,}3861x_4 - 2{,}5094x_5 + 3{,}0709x_6 - 0{,}4934x_7 - 2{,}5824x_8$$

En nuestro caso, Lasso ayudó a identificar las variables más relevantes para predecir el precio de las habitaciones, mientras que Ridge observamos que es prácticamente idéntica a la regresión lineal simple.

Regresión K-Nearest Neighbors (KNN):

Este modelo es un enfoque no paramétrico que predice el valor de la variable objetivo basándose en los k vecinos más cercanos en el espacio de características. En el caso de la predicción del precio medio por habitación, este modelo considera que las habitaciones con características similares tendrán precios similares.

Con este modelo somos capaces de captar relaciones complejas y no lineales en los datos, lo que lo hace muy flexible.

Árboles de Decisión:

Dividen los datos en subconjuntos mediante reglas, capturando relaciones complejas y no lineales. En cada nodo del árbol, los datos se dividen de manera que se minimice una métrica de impureza, como el error cuadrático medio (MSE) para problemas de regresión.

Los árboles de decisión son útiles en nuestro caso porque capturan tanto relaciones lineales como no lineales en los datos.

Random Forest:

Ajusta una serie de regresores de árboles de decisión en varias submuestras del conjunto de datos y utiliza promedios para mejorar la precisión predictiva y controlar el sobreajuste. Cada árbol se ha entrenado utilizando un subconjunto generado por 'Bootstrapping', combinando sus predicciones. Aunque son más difíciles de interpretar, los Bosques nos permiten evitar el sobreajuste a los datos y capturar relaciones más complejas entre estos. Se han modelado 100 arboles con un mínimo de 2 muestras necesarias para dividir un nodo interno y una profundidad máxima del árbol de 10 nodos.

Métricas de Evaluación

Para evaluar el rendimiento de los modelos, hemos utilizado exclusivamente métricas de regresión, ya que el objetivo es predecir el precio medio por habitación. Las métricas empleadas fueron:

- **Error Cuadrático Medio (MSE):** Mide la magnitud promedio de los errores cuadráticos. Es útil para penalizar errores grandes y evaluar la precisión general del modelo.
- **Raíz del Error Cuadrático Medio (RMSE):** Es la raíz cuadrada del MSE y proporciona una interpretación más directa del error en las mismas unidades que la variable objetivo (precios de las habitaciones).
- **R^2 (Coeficiente de Determinación):** Evalúa qué proporción de la varianza en la variable objetivo es explicada por el modelo. Es una métrica estándar para medir la calidad del ajuste.

Estas métricas nos permitieron comparar de manera objetiva el desempeño de los distintos modelos, seleccionando aquellos que equilibraron precisión y simplicidad.

5. Resultados e Interpretación

5.1. Presentación de los resultados obtenidos con cada uno de los modelos

- **Regresión lineal**: Este modelo simple ha obtenido un RMSE de **30.79** en el conjunto de prueba y **30.87** en el conjunto de entrenamiento. Esto indica un rendimiento consistente y ninguna evidencia clara de sobreajuste o subajuste. Los coeficientes obtenidos permiten una interpretación directa del impacto de cada variable en la predicción.

- **Regresión Ridge y Lasso**: Ajustando estos modelos a nuestros datos, hemos obtenido los siguientes resultados:

Modelo	α	RMSE (Test)	RMSE (Train)
Lasso	0.001	30.7935	30.8684
	0.01	30.7933	30.8684
	0.1	30.7925	30.8695
	0.5	30.8114	30.8945
	1.0	30.8823	30.9659
	5.0	31.8568	32.0774
	10.0	33.9876	34.3711
Ridge	0.001	30.7935	30.8684
	0.01	30.7935	30.8684
	0.1	30.7935	30.8684
	0.5	30.7935	30.8684
	1.0	30.7935	30.8684
	5.0	30.7935	30.8684
	10.0	30.7934	30.8684

Cuadro 1: Resultados de RMSE para los modelos de Lasso y Ridge con diferentes valores de α.

Con valores pequeños de α (por ejemplo, 0.001 y 0.01), los resultados de Ridge y Lasso son casi idénticos a los de la regresión lineal. Al aumentar α, Lasso redujo algunos coeficientes a 0, seleccionando las variables más importantes, mientras que Ridge mantiene todos los coeficientes reducidos en magnitud.

  - RMSE Ridge (prueba): **30.79** (para α pequeño).
  - RMSE Lasso (prueba): **30.79** (para α pequeño) y aumenta para valores grandes de α.

- **K-Nearest Neighbors (KNN)**: Estos son los resultados del modelo KNN aplicado a nuestros datos:

k	RMSE (Test)	RMSE (Train)
1	35.4105	25.8009
2	30.7375	23.7391
3	29.4434	24.1180
5	28.3913	24.6310
10	27.7514	25.6690

Cuadro 2: Resultados de RMSE para diferentes valores de vecinos (k) en el modelo KNN.

Este modelo no paramétrico ha obtenido un mejor rendimiento con valores de $k = 5$ y $k = 10$, con RMSE de **28.39** y **27.75**, respectivamente, en el conjunto de prueba. Sin embargo, para $k = 1$ o $k = 2$, el modelo muestra signos de sobreajuste, con un RMSE de **35.41** (prueba) y un RMSE de entrenamiento mucho menor (25.80).

- **Árboles de decisión**: Los resultados de los árboles dependen de la profundidad y la configuración del parámetro `min samples split`. Estos son los resultados obtenidos:

  Para una profundidad de 10 y valores de `min samples split` entre 2 y 9, el RMSE en prueba se sitúa en **28.80**, mostrando un buen equilibrio entre complejidad y rendimiento. Sin embargo,

Depth	*Min Samples Split*	RMSE (Test)	RMSE (Train)
2	2-9	30.6507	30.7690
5	2-9	29.4769	29.3111
10	2-9	28.8544-28.6754	27.5690-27.7091
25	2-9	31.3397-29.4359	19.7335–22.4104
44	2-9	31.5917-29.4480	18.7238–21.8365

Cuadro 3: Resultados de RMSE para los modelos de Árbol de Decisión con diferentes profundidades (*Depth*) y tamaños mínimos de división (*Min Samples Split*).

profundidades excesivas (por ejemplo, 25 o más) causan sobreajuste con grandes diferencias entre los errores de entrenamiento y prueba.

- **Random forest**: Este modelo obtuvo un RMSE de **27.29** en el conjunto de entrenamiento y **27.61** en el conjunto de prueba, con valores de R^2 de 0.3971 y 0.3619, respectivamente. Esto indica un buen equilibrio entre entrenamiento y prueba, mostrando una mejora respecto a los árboles de decisión individuales y una capacidad razonable para capturar patrones en los datos.

5.2. Comparación de los modelos en términos de rendimiento

Basado en el RMSE:

- **Mejor modelo**: El modelo KNN con $k = 10$ ha obtenido el RMSE más bajo (27.75), seguido del KNN con $k = 5$ (28.39) y los árboles de decisión con profundidad 10 (28.80).
- **Modelos lineales**: La regresión lineal, Ridge y Lasso con valores pequeños de α tienen un rendimiento consistente con RMSE alrededor de 30.79.
- **Peor comportamiento**: Los árboles con gran profundidad y los modelos KNN con valores pequeños de k mostraron claros signos de sobreajuste.

5.3. Interpretación de los resultados e implicaciones en el contexto del conjunto de datos

Los resultados sugieren que las relaciones entre las variables no son completamente lineales, ya que los modelos no paramétricos (KNN y árboles de decisión) superan a los métodos lineales en términos de rendimiento. El modelo KNN con $k = 10$ es especialmente adecuado para este conjunto de datos, ya que captura relaciones locales sin sobreajustarse.

Las técnicas de regularización (Ridge y Lasso) pueden ser útiles para manejar la multicolinealidad y seleccionar variables importantes, haciéndolas adecuadas cuando la interpretabilidad sea prioritaria. Por otro lado, los árboles de decisión han destacado por su capacidad de interpretación y por capturar patrones no lineales, aunque con cierto riesgo de sobreajuste.

6. Conclusiones y Futuro Trabajo

6.1. Interpretación conceptual de las variables y su impacto en el precio del hotel

Interpretación desde Ridge

- **Número de adultos (coeficiente: +2.93):** Este es el predictor más importante en Ridge. Un mayor número de adultos en la reserva incrementa significativamente el precio promedio por habitación.
- **Número de niños (coeficiente: -2.04):** Impacto negativo. Los hoteles podrían ofrecer descuentos para familias.

- **Antelación de la reserva (coeficiente: -0.09):** Reservas anticipadas se benefician de descuentos.
- **Noches fin de semana (coeficiente: +0.03) y noches entre semana (coeficiente: +0.63):** Impacto positivo moderado.

Interpretación desde el Árbol de Decisión

- **Número de adultos (importancia: 0.71):** Variable más importante. Los precios aumentan con más adultos.
- **Número de niños (importancia: 0.23):** Importancia significativa, aunque menor que la de adultos.
- **Noches entre semana y fines de semana:** Impacto relevante pero menor.
- **Antelación de la reserva (importancia: 0.003):** Efecto mínimo.

6.2. Conclusiones generales sobre el uso de aprendizaje automático

- Los modelos no paramétricos capturan relaciones no lineales y son útiles cuando la interpretación no es prioridad.
- Los modelos lineales y regularizados (Ridge y Lasso) son efectivos para interpretar y manejar multicolinealidad.
- La selección adecuada del modelo depende del objetivo del análisis y las características de los datos.

6.3. Limitaciones del trabajo y posibles futuras mejoras

- Usar modelos más avanzados como Gradient Boosting podría capturar patrones más complejos.
- Optimizar modelos con validación cruzada o búsqueda de hiperparámetros.

6.4. Propuestas de trabajo futuro

- Implementar Gradient Boosting para equilibrar interpretabilidad y precisión.
- Aplicar aprendizaje no supervisado como clustering para segmentar clientes.

7. Anexo

7.1. Codigo en Python

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Cargar datos
df = pd.read_csv('Hotel Reservations.csv', sep=',')

# Visualizar las primeras filas
df.head()

# Tipos de datos
df.dtypes
# int = números enteros (contar casos)
# float = números decimales (hacer cálculos)
# object = texto, fechas, listas... (datos sin procesar)

# Convertir columnas a categoría
categorical_cols = ['type_of_meal_plan', 'market_segment_type', 'room_type_reserved', 'booking_status
df[categorical_cols] = df[categorical_cols].astype('category')

df['repeated_guest'] = df['repeated_guest'].astype('bool')
df['required_car_parking_space'] = df['required_car_parking_space'].astype('bool')

# Columnas numéricas
numeric_cols = ['no_of_adults', 'no_of_children', 'no_of_week_nights', 'no_of_weekend_nights', 'lead_

df.dtypes

# Estadística descriptiva
print(df.describe())
print(df.describe(include='object'))

# Identificación de valores perdidos
print(df.isnull().sum())
print(df.isna().sum().sum())

# Estadísticas de columnas numéricas
for col in numeric_cols:
    print(f"{col}:")
    print(f"Media: {df[col].mean()}")
    print(f"Mediana: {df[col].median()}")
    print(f"Desviación Estándar: {df[col].std()}")
    print("\n")

# Histogramas para variables numéricas
analizar_cols = ['lead_time', 'no_of_adults', 'avg_price_per_room', 'booking_status', 'market_segment
for col in analizar_cols:
    if df[col].dtypes in ['int64', 'float64']:
        plt.hist(df[col], bins=20, edgecolor='k')
        plt.title(f'Distribución de {col}')
        plt.xlabel(col)
        plt.ylabel('Frecuencia')
```

```
        plt.show()

# Frecuencia para variables categóricas
c = ['booking_status', 'market_segment_type', 'room_type_reserved']
for col in c:
    sns.countplot(x=col, data=df)
    plt.title('Frecuencia')
    plt.xticks(rotation=45)
    plt.show()

# Matriz de correlación
analizar_numeric = ['lead_time', 'no_of_adults', 'avg_price_per_room', 'no_of_previous_cancellations'
correlation_matrix = df[analizar_numeric].corr()

plt.figure(figsize=(10, 8))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm')
plt.title('Matriz de Correlación')
plt.show()

# Scatterplot
sns.scatterplot(x='lead_time', y='avg_price_per_room', hue='booking_status', data=df)
plt.title('Relación entre Lead Time y Precio Promedio por Habitación')
plt.xlabel('Lead Time')
plt.ylabel('Precio Promedio por Habitación')
plt.show()

# Identificación de valores atípicos
for col in analizar_numeric:
    sns.boxplot(data=df[col])
    plt.title(f'Detección de Valores Atípicos en {col}')
    plt.show()

# Normalización
for c in numeric_cols:
    if c != 'avg_price_per_room':
        df[f"{c}_scaled"] = (df[c] - df[c].mean()) / df[c].std()

explanatory_variables = [c for c in df.columns if "_scaled" in c]
target_variable = 'avg_price_per_room'

# División en train y test
from sklearn.model_selection import train_test_split
X = df[explanatory_variables]
y = df[target_variable]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=1)

# Modelos de regresión lineal, Lasso, Ridge y evaluación

model = LinearRegression().fit(X_train, y_train)
y_pred = model.predict(X_test)
y_pred_train = model.predict(X_train)
mse_test = mean_squared_error(y_test, y_pred)
rmse_test = np.sqrt(mse)
mse_train = mean_squared_error(y_train, y_pred_train)
rmse_train = np.sqrt(mse_train)
```

```
# Regresión con Lasso:

def lasso_regression(alpha):
    model = Lasso(alpha=alpha).fit(X_train, y_train)
    y_pred = model.predict(X_test)
    y_pred_train = model.predict(X_train)
    mse_test = mean_squared_error(y_test, y_pred)
    mse_train = mean_squared_error(y_train, y_pred_train)
    rmse_test=np.sqrt(mse_test)
    rmse_train=np.sqrt(mse_train)
    return rmse_test,rmse_train

alphas = [0.001, 0.01, 0.1, 0.5, 1.0, 5.0, 10.0]
mse_test_L = []
mse_train_L = []
for alpha in alphas:
  rmse_test_L,rmse_train_L = lasso_regression(alpha)
  mse_test_L.append(rmse_test_L)
  mse_train_L.append(rmse_train_L)


# Regresión con Ridge:

def ridge_regression(alpha):
    model = Ridge(alpha=alpha).fit(X_train, y_train)
    y_pred = model.predict(X_test)
    y_pred_train = model.predict(X_train)

    mse_test = mean_squared_error(y_test, y_pred)
    mse_train = mean_squared_error(y_train, y_pred_train)
    rmse_test=np.sqrt(mse_test)
    rmse_train=np.sqrt(mse_train)
    return rmse_test,rmse_train

mse_test_R = []
mse_train_R = []
for alpha in alphas:
  rmse_test_R,rmse_train_R = ridge_regression(alpha)
  mse_test_R.append(rmse_test_R)
  mse_train_R.append(rmse_train_R)

# Regresión KNN:

def knn_regression(k):
    model = KNeighborsRegressor(n_neighbors=k).fit(X_train, y_train)
    y_pred = model.predict(X_test)
    y_pred_train = model.predict(X_train)
    mse_test = mean_squared_error(y_test, y_pred)
    mse_train = mean_squared_error(y_train, y_pred_train)
    rmse_test=np.sqrt(mse_test)
    rmse_train=np.sqrt(mse_train)

    return rmse_test,rmse_train

k_values = [1, 2, 3, 5, 10]
```

```
mse_test_Knn = []
mse_train_Knn = []
for k in K_neighbor:
   rmse_test_Knn,rmse_train_Knn = model_knn_regressor(k)
   mse_test_Knn.append(rmse_test_Knn)
   mse_train_Knn.append(rmse_train_Knn)

#Arbol de decisión:
def DT_regression(md,ms):
    model = DecisionTreeRegressor(max_depth=md, min_samples_split=ms).fit(X_train, y_train)
    y_pred_train = model.predict(X_train)
    y_pred_test = model.predict(X_test)
    mse_test = mean_squared_error(y_test, y_pred_test)
    mse_train = mean_squared_error(y_train, y_pred_train)
    rmse_test=np.sqrt(mse_test)
    rmse_train=np.sqrt(mse_train)
    md1 = model.get_depth()

    print("Model Depth: {}".format(md1))
    print("Min Samples Split: {}".format(ms))
    print("\tRMSE_TEST: {:.4f}".format(rmse_test))
    print("\tRMSE_TRAIN: {:.4f}".format(rmse_train))

max_depths = [2,5,10,25,50]
min_sample_splits = range(2,10)
for MD in max_depths:
    for MS in min_sample_splits:
        DT_regression(MD,MS)

# Random Forest:

def rf_regression(n_estimators=100, max_depth=10, min_samples_split=2):
    model = RandomForestRegressor(n_estimators=n_estimators, max_depth=max_depth, min_samples_split=m
    y_pred_train = model.predict(X_train)
    y_pred_test = model.predict(X_test)

    mse_train = mean_squared_error(y_train, y_pred_train)
    mse_test = mean_squared_error(y_test, y_pred_test)
    rmse_train = np.sqrt(mse_train)
    rmse_test = np.sqrt(mse_test)
    r2_train = r2_score(y_train, y_pred_train)
    r2_test = r2_score(y_test, y_pred_test)

    print("Random Forest para Regresión")
    print(f"RMSE Entrenamiento: {rmse_train:.4f}")
    print(f"RMSE Prueba: {rmse_test:.4f}")
    print(f"R2 Entrenamiento: {r2_train:.4f}")
    print(f"R2 Prueba: {r2_test:.4f}")

    return model

rf_model = RF_regression(n_estimators=100, max_depth=10, min_samples_split=2)
```